



SPRING-BOOT 3.5 → 4.0 · MINI GUIDE



Jackson & JSON: 3.5 → 4.0

Nine JSON changes everyone hits

One cut from the full Spring Break Cheat Sheets. Every card verified against Spring Boot 3.5.16 and 4.0.7. Real error output, real fixes, from the spring-break test suite. Steve Poole.

About this mini guide

This is one cut from **Spring Break Cheat Sheets**, the field guide to every breaking change between Spring Boot 3.5 and 4.0. Each card covers one change, verified by running real code on 3.5.16 and 4.0.7. One change, one card, one page: the error you'll see, the fix, and the traps.

HOW TO READ A CARD

TIER	How it fails. Tier 1 won't build, Tier 2 won't start, Tier 3 behaves differently. Tier 3 is the dangerous one: nothing goes red, your data just changes shape.
IMPACT	Effort per occurrence once found. S is find-and-replace, M needs an hour of care, L needs a plan.
OPENREWRITE	The badge means the Boot 4.0 recipe fixes this one for you. Run it before touching anything by hand.
WHAT YOU'LL SEE	The real output, so you can match it against your own build.
THE FIX	Before and after code, plus the steps when a rename isn't enough.
WATCH OUT	The traps that make this change look like a different bug.
VERIFY	The command that proves you're done.

THE SERIES

Spring Batch: 3.5 → 4.0	Eight changes between your jobs and 4.0
Hibernate & JPA: 3.5 → 4.0	Ten data-layer changes, mostly the quiet kind
Security & SSO: 3.5 → 4.0	Seven auth changes you cannot ignore
Data & Messaging: 3.5 → 4.0	Nine changes across your stores and streams
Testing: 3.5 → 4.0	Seven test-suite breaks, one clean build away
Observability & Ops: 3.5 → 4.0	Six changes your dashboards will notice first
Jackson & JSON: 3.5 → 4.0	Nine JSON changes everyone hits THIS GUIDE
Web & HTTP: 3.5 → 4.0	Eight changes at the HTTP layer
Core Platform & Build: 3.5 → 4.0	Twenty platform changes underneath everything
The Big Jobs: L-Impact Changes	The six changes that eat your sprint

The full guide collects every change in one book, with a complete planning matrix and the OpenRewrite robot list. github.com/spoole167/spring-break



About HeroDevs

HeroDevs keeps critical open source software secure after the community stops patching it. Never-Ending Support (NES) covers CVEs in Spring Boot 3.5, Java, and dozens of other end-of-life frameworks, so your team migrates on its own schedule instead of the vendor's deadline.

This book exists because migrations fail from surprise more often than difficulty. Every card is a fact, not a guess: verified by running real code on Spring Boot 3.5.16 and 4.0.7.

The complete book, every subsystem in one volume, is coming to LeanPub. Watch github.com/spoole167/spring-break for the link.

herodevs.com

The Planning Matrix

Every change in this book, in book order. Impact is the effort per occurrence once found (S small, M moderate, L large), not how many you have. Auto means the OpenRewrite Boot 4.0 recipe handles it.

9 CHANGES

3 × TIER 1 - WON'T BUILD

2 × TIER 2 - WON'T START

4 × TIER 3 - BEHAVES DIFFERENTLY

TIER	CHANGE	IMPACT	AUTO
1	Jackson Class Renames	M	Y
1	@JsonComponent and @JsonMixin Renamed to @JacksonComponent and @JacksonMixin	S	Y
1	Jackson 3.0 Group ID Change	L	Y
2	write-dates-as-timestamps Property Breaks Startup	M	•
2	JacksonException No Longer Extends IOException	M	Y
3	Jackson Date Serialisation Flip	M	•
3	Jackson Locale Format Change (BCP 47)	S	•
3	Jackson 3 Auto-Discovers All Modules on Classpath	S	•
3	spring.jackson.default-property-inclusion Silently Ignored	M	•

The Robot List vs The Human List

The left column is what the OpenRewrite Boot 4.0 recipe fixes while you get coffee. The right column needs a person who understands your code. This split is the honest size of the migration: the renames are free, the behaviour changes are the work.

THE ROBOT LIST

Run the recipe, review the diff, done.

Jackson Class Renames

@JsonComponent and @JsonMixin Renamed to
@JacksonComponent and @JacksonMixin

Jackson 3.0 Group ID Change

JacksonException No Longer Extends IOException

THE HUMAN LIST

Read the card. Check your code. Decide.

write-dates-as-timestamps Property Breaks Startup

Jackson Date Serialisation Flip

Jackson Locale Format Change (BCP 47)

Jackson 3 Auto-Discovers All Modules on Classpath

spring.jackson.default-property-inclusion Silently Ignored

TIER 1

Tier 1 - Won't Build

3 cards

1. Jackson Class Renames
2. @JsonComponent and @JsonMixin Renamed to @JacksonComponent and @JacksonMixin
3. Jackson 3.0 Group ID Change

Jackson Class Renames

Tier 1 - Won't Build

Jackson & JSON

Impact: M

🔄 OpenRewrite

WHAT CHANGED

Jackson 3.0 renamed several core classes: `JsonSerializer` became `ValueSerializer`, `JsonDeserializer` became `ValueDeserializer`, `SerializerProvider` became `SerializationContext`, `DeserializationContext` kept its name but moved packages. Method signatures on these classes also changed.

WHY

The "Json" prefix was misleading: Jackson handles YAML, XML, CBOR, and other formats, so serializers work with values, not specifically JSON. The rename also aligned the class hierarchy with the new streaming API.

SCOPE CHECK

Search for `extends JsonSerializer`, `extends JsonDeserializer`, `SerializerProvider`, and `JsonMappingException`. Every custom serializer, deserializer, and exception handler needs updating.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/DateSerializer.java:[4,44]
error: cannot find symbol
  symbol:   class JsonSerializer
  location: package tools.jackson.databind
[ERROR] /src/main/java/com/example/DateSerializer.java:[12,47]
error: cannot find symbol
  symbol:   class SerializerProvider
  location: package tools.jackson.databind
```

THE FIX

Custom serializer class

BEFORE

```
import com.fasterxml.jackson.databind.JsonSerializer;
import com.fasterxml.jackson.databind.SerializerProvider;

public class DateSerializer extends JsonSerializer<LocalDate> {
    @Override
    public void serialize(LocalDate value, JsonGenerator gen,
        SerializerProvider provider) throws IOException {
```


AFTER

```
import tools.jackson.databind.ser.ValueSerializer;
import tools.jackson.databind.SerializationContext;

public class DateSerializer extends ValueSerializer<LocalDate> {
    @Override
    public void serialize(LocalDate value, JsonGenerator gen,
        SerializationContext ctxt) throws IOException {
```

Custom deserializer class

BEFORE

```
import com.fasterxml.jackson.databind.JsonDeserializer;

public class DateDeserializer extends JsonDeserializer<LocalDate> {
```

AFTER

```
import tools.jackson.databind.deser.ValueDeserializer;

public class DateDeserializer extends ValueDeserializer<LocalDate> {
```

Exception handling

BEFORE

```
import com.fasterxml.jackson.databind.JsonMappingException;
```

AFTER

```
import tools.jackson.databind.DatabindException;
```

HOW TO FIX

OpenRewrite (recommended).

Run the [Jackson 3 type changes recipe](#). It handles class renames, method signature updates, and import changes in one pass.

Manual migration.

Replace class names and imports file by file. The full mapping is in the [Jackson 3 migration guide](#). Don't forget to update method parameter types in overridden methods.

WATCH OUT

The method signatures changed too. `serialize()` now takes a `SerializationContext` instead of `SerializerProvider`. Renaming the class but keeping the old parameter type gives an abstract method error instead of a clean override.

Annotations like `@JsonSerialize(using = ...)` still work but must reference the new class names. A custom serialiser registered via annotation will fail at runtime if it extends the wrong base class.

FURTHER INFO

Driven by Jackson 3.0, upstream of Spring Boot 4.0, and finalised in Jackson 3.0-rc1. Affects spring-boot-starter-json consumers with custom serialisers. See also: [jackson-group-id](#), [jackson-exception-hierarchy](#).

@JsonComponent and @JsonMixin Renamed to @JacksonComponent and @JacksonMixin

Tier 1 - Won't Build

Jackson & JSON

Impact: S

🔄 OpenRewrite

`@JsonComponent` and `@JsonMixin` are removed in Boot 4.0 and fail to compile. Replace with `@JacksonComponent` and `@JacksonMixin`.

WHAT CHANGED

Spring Boot's custom serializer registration annotations were renamed to align with Jackson 3's branding. `@JsonComponent` becomes `@JacksonComponent` and `@JsonMixin` becomes `@JacksonMixin`. The functionality is identical: only the annotation names changed.

WHY

Jackson 3 rebranded from the `com.fasterxml.jackson` group ID to `tools.jackson`. Spring Boot's annotation names were updated to match, making it clear they belong to the Jackson 3 ecosystem rather than the legacy Jackson 2 stack.

SCOPE CHECK

Grep for `@JsonComponent` and `@JsonMixin` across your source tree. Also search for `import org.springframework.boot.jackson.JsonComponent` and `import org.springframework.boot.jackson.JsonMixin`.

WHAT YOU'LL SEE

```
@JsonComponent
public class MoneySerializer extends JsonSerializer<Money> { ... }

// Boot 4.0 compile error:
error: cannot find symbol
  symbol: @JsonComponent
  location: class MoneySerializer
```

THE FIX

Custom serializer

BEFORE

```
import org.springframework.boot.jackson.JsonComponent;

@JsonComponent
public class MoneySerializer extends JsonSerializer<Money> { ... }
```

AFTER

```
import org.springframework.boot.jackson.JacksonComponent;

@JacksonComponent
public class MoneySerializer extends JsonSerializer<Money> { ... }
```

Mixin

BEFORE

```
import org.springframework.boot.jackson.JsonMixin;

@JsonMixin(Money.class)
public abstract class MoneyMixin { ... }
```

AFTER

```
import org.springframework.boot.jackson.JacksonMixin;

@JacksonMixin(Money.class)
public abstract class MoneyMixin { ... }
```

HOW TO FIX

Rename the annotations.

Replace all `@JsonComponent` with `@JacksonComponent` and all `@JsonMixin` with `@JacksonMixin`. Update the imports accordingly.

VERIFY

Custom serialisers and mixins annotated with the new names register correctly with the auto-configured `ObjectMapper`

WATCH OUT

Do not confuse Spring Boot's `@JacksonComponent` with Jackson's own `@JsonSerialize` / `@JsonDeserialize` annotations. Spring Boot's annotation auto-registers the serialiser with the `ObjectMapper` bean; Jackson's own annotations require explicit wiring.

FURTHER INFO

`@JsonComponent` and `@JsonMixin` were Spring Boot-specific annotations that registered custom serialisers, deserialisers, and mixins with the auto-configured `ObjectMapper`.

spring-break module: `jackson-component-rename` [Spring Boot 4.0 Migration Guide](#)

Jackson 3.0 Group ID Change

Tier 1 - Won't Build

Jackson & JSON

Impact: L

🔄 OpenRewrite

WHAT CHANGED

Jackson 3.0 moved from the `com.fasterxml.jackson` Maven group to `tools.jackson`. Spring Boot 4.0's managed dependencies point to Jackson 3.0, so the old group ID no longer resolves. Every import statement using `com.fasterxml.jackson` breaks.

WHY

Jackson's maintainers separated the project identity from the original FasterXML organisation, giving a clean break to drop deprecated APIs and ship Jackson 3.0 as a proper major release.

SCOPE CHECK

Search your codebase for `com.fasterxml.jackson`. Every hit is a coordinate or import that needs updating. If you use `spring-boot-starter-json` or `spring-boot-starter-web`, Jackson arrives transitively. You still need to update explicit dependency declarations and all source-level imports.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/JacksonConfig.java:[3,32]
package com.fasterxml.jackson.databind does not exist
[ERROR] /src/main/java/com/example/JacksonConfig.java:[8,5]
cannot find symbol
symbol: class ObjectMapper
```

THE FIX

pom.xml — explicit Jackson dependency

BEFORE

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
```

AFTER

```
<dependency>
  <groupId>tools.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
```

BEFORE

```
import com.fasterxml.jackson.databind.ObjectMapper;  
import com.fasterxml.jackson.core.JsonProcessingException;
```

AFTER

```
import tools.jackson.databind.ObjectMapper;  
import tools.jackson.core.JsonException;
```

HOW TO FIX

OpenRewrite (recommended).

Run the [Jackson 3 migration recipe](#). Handles group ID changes and import updates across the entire codebase in one pass.

Manual: update Maven coordinates first.

Replace `com.fasterxml.jackson` with `tools.jackson` in all POM or Gradle files, then fix imports file by file.

WATCH OUT

The class renames (JsonSerializer → ValueSerializer, SerializerProvider → SerializationContext) arrive in the same Jackson 3.0 upgrade. Fixing only the group ID leaves broken class references. See [jackson-class-renames](#) for the full list.

Jackson exceptions (`JsonMappingException` , `JsonProcessingException`) are `Serializable` . If you've persisted these anywhere (dead letter queues, error logs), those serialised streams contain the old class names and won't deserialise correctly after migration.

FURTHER INFO

Driven by Jackson 3.0, upstream of Spring Boot 4.0. The FasterXML to tools.jackson rename was announced in 2023 and finalised in Jackson 3.0-rc1. See also: [jackson-class-renames](#), [jackson-exception-hierarchy](#).

spring-break module: [jackson-group-id](#) [Jackson 3 migration guide](#)

TIER 2

Tier 2 - Won't Start

2 cards

1. write-dates-as-timestamps Property Breaks
Startup

2. JacksonException No Longer Extends
IOException

write-dates-as-timestamps Property Breaks Startup

Tier 2 - Won't Start

Jackson & JSON

Impact: M

The `spring.jackson.serialization.write-dates-as-timestamps` property no longer binds in Boot 4.0: Jackson 3 moved the feature out of `SerializationFeature`. One leftover config line stops your application from starting.

WHAT CHANGED

Jackson 3 reorganised its feature flags: date/time toggles such as `WRITE_DATES_AS_TIMESTAMP` are no longer constants on `SerializationFeature`. Boot 4.0 still binds `spring.jackson.serialization.*` keys against that enum, so the old property value fails enum conversion and property binding aborts context startup.

WHY

The old `SerializationFeature` enum values don't map one-to-one to Jackson 3's new configuration API; the date/time features moved to their own feature set. Spring Boot's property binding follows the new enum, so values naming removed constants fail fast instead of being remapped.

SCOPE CHECK

Search your `application.properties` and `application.yml` files for any `spring.jackson.serialization` or `spring.jackson.deserialization` entries. Every one of those feature toggles is potentially dead in Boot 4.0.

WHAT YOU'LL SEE

```
# application.properties (worked on Boot 3.5)
spring.jackson.serialization.write-dates-as-timestamps=true

// Spring Boot 4.0 — application fails to start:
org.springframework.boot.context.properties.bind.BindException:
  Failed to bind properties under 'spring.jackson.serialization' to
  java.util.Map<tools.jackson.databind.SerializationFeature, Boolean>
Caused by: java.lang.IllegalArgumentException: No enum constant
  tools.jackson.databind.SerializationFeature.write-dates-as-timestamps
```

THE FIX

`application.properties` — old (silently ignored)

BEFORE

```
spring.jackson.serialization.write-dates-as-timestamps=true
```

AFTER

```
# Removed — configure via ObjectMapper customiser instead
```

ObjectMapper customiser replacement

BEFORE

```
# (relied on spring.jackson.serialization.write-dates-as-timestamps)
```


AFTER

```
@Bean
public Jackson2ObjectMapperBuilderCustomizer timestampDates() {
    return builder -> builder.featuresToEnable(
        SerializationFeature.WRITE_DATES_AS_TIMESTAMPS
    );
}
```

Alternative: per-field annotation

BEFORE

```
private Instant timestamp;
```

AFTER

```
@JsonFormat(shape = JsonFormat.Shape.NUMBER)
private Instant timestamp;
```

HOW TO FIX

Register an ObjectMapper customiser bean.

Create a `Jackson2ObjectMapperBuilderCustomizer` that explicitly enables `WRITE_DATES_AS_TIMESTAMPS`. This replaces the dead properties-based config.

Migrate clients to ISO-8601 (recommended).

If you can coordinate with API consumers, drop the timestamp format entirely and adopt ISO-8601 strings. Remove the old property and let Jackson 3's new default take effect.

VERIFY

Application fails to start: `BindException` on `spring.jackson.serialization`, "No enum constant `tools.jackson.databind.SerializationFeature.write-dates-as-timestamps`"

WATCH OUT

This compounds with the `jackson-date-format` change. If you set `write-dates-as-timestamps=true` specifically to keep numeric timestamps, you now have two problems: the default flipped, and the property that pinned the old behaviour stops your app from starting.

Any `spring.jackson.serialization.` or `spring.jackson.deserialization.` value naming an enum constant that Jackson 3 moved or removed fails startup the same way.

FURTHER INFO

Driven by the combination of Jackson 3.0 and Spring Boot 4.0's auto-configuration changes. Verified 15 July 2026 on 4.0.7. Compounds with `jackson-date-format`. See also: `jackson-date-format`, `jackson-property-inclusion`.

[spring-break module: jackson-dates-timestamps](#) [Spring Boot 4.0 Migration Guide](#) [Jackson 3 in Spring \(blog\)](#)

JacksonException No Longer Extends IOException

Tier 2 - Won't Start

Jackson & JSON

Impact: M

🔄 OpenRewrite

WHAT CHANGED

In Jackson 2.x, `JacksonException` extended `java.io.IOException`, so any `catch (IOException e)` block would also catch malformed-JSON errors. In Jackson 3.0, `JacksonException` extends `RuntimeException`. Catch blocks for `IOException` no longer intercept Jackson parsing failures, and methods that declared `throws IOException` for Jackson work now throw unchecked exceptions instead.

WHY

The original hierarchy was a design mistake: it forced checked-exception handling even for in-memory parsing, which involves no I/O. Making `JacksonException` unchecked aligns with modern Java conventions and removes the boilerplate `try/catch`.

SCOPE CHECK

Search for `catch (IOException e)`, `catch (JsonProcessingException e)`, and `throws IOException` near any `ObjectMapper` usage. Every hit where the catch was meant to intercept Jackson errors now misses at runtime. REST controllers, message listeners, and integration test assertions are the most common locations.

WHAT YOU'LL SEE

```
com.example.OrderController: Unhandled exception in /api/orders
tools.jackson.core.exc.StreamReadException: Unexpected character ('x' (code 120)):
  expected a valid value (JSON String, Number, Array, Object or token)
  at [Source: (String)"x"; line: 1, column: 2]
  at tools.jackson.core.json.JsonParser._reportUnexpectedChar(JsonParser.java:...)
  ...
Caused by: tools.jackson.core.JsonException (not an IOException!)
---
HTTP/500 Internal Server Error
{"timestamp":"2026-04-15T10:22:03","status":500,
 "error":"Internal Server Error","path":"/api/orders"}
```

THE FIX

Controller error handling — before

BEFORE

```
try {
    Order order = objectMapper.readValue(body, Order.class);
} catch (IOException e) {
    return ResponseEntity.badRequest().body("Invalid JSON");
}
```

AFTER

```
try {  
    Order order = objectMapper.readValue(body, Order.class);  
} catch (JacksonException e) {  
    return ResponseEntity.badRequest().body("Invalid JSON");  
}
```

Spring @ExceptionHandler — before

BEFORE

```
@ExceptionHandler(IOException.class)  
public ResponseEntity<String> handleJsonError(IOException ex) {
```

AFTER

```
@ExceptionHandler(JacksonException.class)  
public ResponseEntity<String> handleJsonError(JacksonException ex) {
```

Method signature cleanup

BEFORE

```
public Order parseOrder(String json) throws IOException {
```

AFTER

```
public Order parseOrder(String json) {
```

HOW TO FIX

Search for catch blocks.

Grep for `catch (IOException` and `catch (JsonProcessingException`. Replace with `catch (JacksonException` where the intent is to handle Jackson parse failures. Keep `IOException` catches for genuine I/O work (file reads, sockets).

Update @ExceptionHandler advice.

If your `@RestControllerAdvice` catches `IOException` to return 400 on bad JSON, add a separate handler for `JacksonException`. Otherwise those errors now surface as unhandled 500s.

Clean up throws clauses.

Methods that declared `throws IOException` only for Jackson can drop the clause entirely. The compiler won't complain, but the stale declaration misleads readers.

WATCH OUT

The code compiles cleanly because `JacksonException` is now unchecked. You only discover the problem when bad input arrives, the catch block no longer fires, and clients get 500 errors instead of 400s.

Spring's built-in `HttpMessageNotReadableException` still wraps Jackson errors for `@RequestBody` parsing. The risk is in manual `ObjectMapper` calls inside your own code.

A broad `catch (Exception` still catches Jackson errors, but loses the ability to return a domain-appropriate error message. Narrow your catch clauses.

FURTHER INFO

Driven by Jackson 3.0, upstream of Spring Boot 4.0. The change was announced in 2022 and finalised in Jackson 3.0-rc1. See also: [jackson-group-id](#), [jackson-class-renames](#).

[spring-break module: jackson-exception-hierarchy](#) [Jackson 3 migration guide](#) [Spring Boot 4.0 Migration Guide](#)

TIER 3

Tier 3 - Behaves Differently

4 cards

1. Jackson Date Serialisation Flip
2. Jackson Locale Format Change (BCP 47)
3. Jackson 3 Auto-Discovers All Modules on Classpath
4. `spring.jackson.default-property-inclusion` Silently Ignored

Jackson Date Serialisation Flip

Tier 3 - Behaves Differently

Jackson & JSON

Impact: M

WHAT CHANGED

Jackson 3.0 changed the default value of `SerializationFeature.WRITE_DATES_AS_TIMESTAMPS` from `true` to `false`. Dates, times, and durations now serialise as ISO-8601 strings by default instead of numeric epoch milliseconds.

WHY

ISO-8601 strings are human-readable, timezone-aware, and the de facto standard in JSON APIs. The old numeric default caused constant confusion about whether the value was seconds or milliseconds, and lost timezone information entirely.

SCOPE CHECK

Search for any DTO field of type `java.util.Date`, `Instant`, `LocalDate`, `LocalDateTime`, `ZonedDateTime`, or `Duration`. Every one of those fields will serialise differently after the upgrade. Check API consumers, contract tests, and front-end parsing code.

WHAT YOU'LL SEE

```
// API response — before (Spring Boot 3.5)
{"created": 1714089600000}

// API response — after (Spring Boot 4.0)
{"created": "2025-04-26T00:00:00Z"}

// Front-end JavaScript blows up
TypeError: date.getTime is not a function
// or: contract test fails
Expected: <1714089600000> (Long)
Actual: <"2025-04-26T00:00:00Z"> (String)
```

THE FIX

application.properties — restore old behaviour

BEFORE

```
# (no explicit setting needed in Boot 3.5)
```

AFTER

```
spring.jackson.serialization.write-dates-as-timestamps=true
```

or adopt ISO-8601 and fix the client

BEFORE

```
const ts = new Date(response.created); // was a number
```

AFTER

```
const ts = new Date(response.created); // now parses ISO string
```

ObjectMapper explicit configuration

BEFORE

```
// Jackson 2 default: timestamps enabled
ObjectMapper mapper = new ObjectMapper();
mapper.registerModule(new JavaTimeModule());
```

AFTER

```
// Jackson 3 default: ISO strings
ObjectMapper mapper = new ObjectMapper();
mapper.registerModule(new JavaTimeModule());
// only if you need the old numeric format:
mapper.enable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
```

HOW TO FIX

Opt in to the new ISO-8601 default (recommended).

Update your API clients and contract tests to expect ISO-8601 strings. This is the better long-term format. Coordinate the change with front-end teams so they parse the string, not a number.

Restore numeric timestamps.

Add `spring.jackson.serialization.write-dates-as-timestamps=true` to `application.properties`. This gives you the old behaviour while you plan the migration.

VERIFY

JSON responses show dates in expected format (check API output)

WATCH OUT

This change is invisible at compile time and at startup. The app runs perfectly until a client tries to parse the date field as a number. Without contract tests, you won't catch this before production.

The `@JsonFormat` annotation on individual fields still overrides the global default. Annotated dates are unaffected; the unannotated ones flip.

FURTHER INFO

Driven by Jackson 3.0, upstream of Spring Boot 4.0, and announced in its release notes. See also: [jackson-dates-timestamps](#), [jackson-property-inclusion](#).

[spring-break module: jackson-date-serialisation](#) [Jackson 3 migration guide](#) [Jackson 3 in Spring \(blog\)](#)

Jackson Locale Format Change (BCP 47)

Tier 3 - Behaves Differently

Jackson & JSON

Impact: S

Jackson 3.0 serialises `Locale` using IETF BCP 47 format (`zh-CN`) instead of Java's underscore format (`zh_CN`). Locale-matching logic silently breaks.

WHAT CHANGED

Jackson 3.0 switched `Locale` serialisation from Java's `toString()` `format ( zh_CN )` to the IETF BCP 47 language tag `format ( zh-CN )`. Underscores become hyphens and the structure follows RFC 5646.

WHY

BCP 47 is the web standard used by HTTP `Accept-Language` headers, HTML `lang` attributes, and JavaScript's `Intl` API. Aligning Jackson's output with the rest of the web ecosystem removes a constant source of mismatch bugs.

SCOPE CHECK

Search for any DTO or entity field of type `java.util.Locale` that gets serialised to JSON. Also check database columns, message queues, and `i18n` resource bundle keys that use the underscore format. Every one of those is a potential mismatch.

WHAT YOU'LL SEE

```
// API response - before (Spring Boot 3.5)
{"userLocale": "zh_CN"}

// API response - after (Spring Boot 4.0)
{"userLocale": "zh-CN"}

// Downstream lookup fails
Map<String, String> labels = i18nMap.get(user.getLocale());
// returns null - map keys use "zh_CN", response now sends "zh-CN"

// Test failure
org.opentest4j.AssertionFailedError:
Expected: "zh_CN"
Actual: "zh-CN"
```

THE FIX

Custom serialiser to restore old format

BEFORE

```
// No custom serialiser needed in Boot 3.5
```

AFTER

```
@JsonSerialize(using = JavaLocaleSerializer.class)
private Locale userLocale;
```

Or update all consumers to use BCP 47

BEFORE

```
String localeKey = user.getLocale(); // "zh_CN"
```


AFTER

```
String localeKey = Locale.forLanguageTag(user.getLocale()).toString(); // normalise
```

Fix i18n lookup maps

BEFORE

```
i18nMap.put("zh_CN", chineseLabels);
```

AFTER

```
i18nMap.put("zh-CN", chineseLabels);
```

HOW TO FIX

Adopt BCP 47 throughout (recommended).

Update your i18n maps, database locale columns, and downstream consumers to use hyphenated BCP 47 tags. This aligns with HTTP headers and browser APIs. Run a data migration for persisted locale strings.

Write a custom Locale serialiser.

Register a custom Jackson serialiser that calls `locale.toString()` instead of `locale.toLanguageTag()`. This preserves the old underscore format while you plan the broader migration.

VERIFY

Locale-sensitive fields render correctly in all target locales

WATCH OUT

If you store locale strings in a database and compare them with equality checks, the format flip causes mismatches that surface as missing data, not errors. A `SELECT WHERE locale = 'zh_CN'` query won't match the new `zh-CN` value sent by the API.

BCP 47 has subtag rules beyond swapping underscores for hyphens. Some locales have three components (e.g. `zh_Hant_TW` becomes `zh-Hant-TW`). Use `Locale.forLanguageTag()` for correct round-tripping.

FURTHER INFO

Driven by Jackson 3.0, upstream of Spring Boot 4.0. See also: [jackson-date-format](#), [jackson-group-id](#).

[spring-break module: jackson-locale-format](#) [Jackson 3 migration guide](#) [IETF BCP 47 \(RFC 5646\)](#)

Jackson 3 Auto-Discovers All Modules on Classpath

Tier 3 - Behaves Differently

Jackson & JSON

Impact: S

Jackson 3 registers every module it finds on the classpath. Modules that arrive as transitive dependencies can silently change your JSON output. Disable with `spring.jackson.find-and-add-modules=false`.

WHAT CHANGED

Jackson 3 uses the Java ServiceLoader mechanism to find and register all `com.fasterxml.jackson.databind.Module` implementations on the classpath. Spring Boot no longer controls which modules are registered: every module present is activated.

WHY

Automatic discovery lets library authors ship Jackson integration without requiring explicit registration. Spring Boot aligned with Jackson 3's convention.

SCOPE CHECK

Run your application's serialisation tests after upgrading. Compare JSON output field by field for any type that uses custom serialisation or has complex types (dates, BigDecimal, Optional, collections). Grep for `ObjectMapper` customisation code to understand what was previously registered manually.

WHAT YOU'LL SEE

```
// Boot 3.5: only well-known modules registered. Output is predictable.
{"amount": 150.00, "currency": "EUR"}

// Boot 4.0: a transitive Jackson module on the classpath changes
// BigDecimal serialization:
{"amount": "150.00", "currency": "EUR"}

// No error, no warning. The JSON schema changed.
```

THE FIX

application.properties — disable auto-discovery to restore Boot 3.5 behaviour

BEFORE

```
# Boot 3.5: no property needed, only well-known modules registered
```

AFTER

```
# Boot 4.0: disable auto-discovery to prevent unexpected module activation
spring.jackson.find-and-add-modules=false
```

HOW TO FIX

Audit classpath modules and disable auto-discovery if needed.

Run `mvn dependency:tree` and look for any artifacts ending in `-module` or containing `jackson`. For each one, verify its effect on your serialisation output. If unexpected modules are activating, set `spring.jackson.find-and-add-modules=false` and register only the modules you need explicitly.

Explicitly register required modules.

After disabling auto-discovery, register required modules via a `Jackson2ObjectMapperBuilderCustomizer` (now renamed `JsonMapperBuilderCustomizer`) bean that calls `modules(new JavaTimeModule(), ...)`.

VERIFY

JSON serialisation output is identical before and after the upgrade, or unexpected module behaviour is controlled with `spring.jackson.find-and-add-modules`

WATCH OUT

This interacts with the Jackson 2 compatibility shim (`spring-boot-jackson2`). If you have both Jackson 2 and Jackson 3 modules on the classpath, both may be auto-discovered and conflict.

Test suites that stub or mock the `ObjectMapper` will not catch this: the change only manifests with a real, auto-configured `ObjectMapper` bean.

FURTHER INFO

In Boot 3.5 (Jackson 2), Spring Boot registered a fixed set of well-known modules: Java time, parameter names, and a handful of others. Everything else needed explicit registration. A module a library brought in for its own internal use can now alter your own `ObjectMapper` output.

spring-break module: `jackson-module-autodiscovery` [Spring Boot 4.0 Migration Guide](#)

spring.jackson.default-property-inclusion Silently Ignored

Tier 3 - Behaves Differently

Jackson & JSON

Impact: M

The `spring.jackson.default-property-inclusion` property no longer configures Jackson 3's ObjectMapper. Null fields reappear in your API responses without warning.

WHAT CHANGED

Spring Boot 4.0's auto-configuration for Jackson 3 no longer reads the `spring.jackson.default-property-inclusion` property. The property sits in your config file doing nothing: no deprecation warning, no startup error. The ObjectMapper reverts to Jackson's default inclusion policy: `ALWAYS` (include everything, including nulls).

WHY

Jackson 3 moved inclusion configuration from a single global enum to a granular per-type system. The old Spring Boot property didn't map cleanly to the new API, so the binding was removed pending a redesign.

SCOPE CHECK

Check whether your `application.properties` or `application.yml` sets `spring.jackson.default-property-inclusion`. If it does, that setting is now dead.

WHAT YOU'LL SEE

```
# application.properties (still present, no startup warning)
spring.jackson.default-property-inclusion=non_null

// API response — before (Spring Boot 3.5)
{"name": "Alice", "age": 30}

// API response — after (Spring Boot 4.0)
{"name": "Alice", "age": 30, "middleName": null, "nickname": null}

// Contract test failure
org.opentest4j.AssertionFailedError:
Expected JSON keys: [name, age]
Actual JSON keys: [name, age, middleName, nickname]
```

THE FIX

`application.properties` — old (silently ignored)

BEFORE

```
spring.jackson.default-property-inclusion=non_null
```

AFTER

```
# Removed — configure via ObjectMapper bean instead
```

ObjectMapper customiser bean

BEFORE

```
# (relied on spring.jackson.default-property-inclusion)
```

AFTER

```
@Bean
public Jackson2ObjectMapperBuilderCustomizer nonNullInclusion() {
    return builder -> builder
        .serializationInclusion(JsonInclude.Include.NON_NULL);
}
```

Or per-class annotation

BEFORE

```
public class UserDto {
```

AFTER

```
@JsonInclude(JsonInclude.Include.NON_NULL)
public class UserDto {
```

HOW TO FIX

Register an ObjectMapper customiser bean (recommended).

Create a `Jackson2ObjectMapperBuilderCustomizer` bean that calls `serializationInclusion(NON_NULL)`. This replaces the property-based configuration and works with Jackson 3.

Annotate DTOs individually.

Add `@JsonInclude(JsonInclude.Include.NON_NULL)` to each DTO class. More verbose, but gives you per-class control and makes the contract explicit in the code.

VERIFY

JSON output excludes null fields where expected (compare before/after)

WATCH OUT

The only signal is larger JSON payloads with unexpected null fields. Without contract tests, that is easy to miss.

If your front-end checks for key presence (e.g. `if ("middleName" in user)`) instead of checking for null values, the extra null fields change its branching logic.

FURTHER INFO

Driven by the combination of Jackson 3.0 and Spring Boot 4.0's auto-configuration changes. See also: [jackson-dates-timestamps](#), [jackson-date-format](#).

[Spring-Break Demo](#) [Spring Boot 4.0 Migration Guide](#) [Jackson 3 in Spring \(blog\)](#)



Not ready to migrate?

Spring Boot 3.5 stops receiving fixes when open-source support ends. You have three sane options. Pick one deliberately.

1. **Pin 3.5 and track CVEs.** Free, but the clock is running and every new CVE is your team's problem to triage.
2. **Migrate incrementally with OpenRewrite.** The Boot 4.0 recipe automates the renames in this book; the behavioural changes still need eyes.
3. **HeroDevs NES for Spring.** Security patches for 3.5 beyond EOL, buying your team time to migrate on your schedule, not the calendar's.

Spring Boot 4.0 Migration Guide • github.com/spring-projects/spring-boot/wiki
OpenRewrite Boot 4.0 recipe • docs.openrewrite.org/recipes/java/spring/boot4
HeroDevs NES for Spring • herodevs.com/support/spring-nes
The test suite behind this book • github.com/spoole167/spring-break